

Table of Contents

EPIC Compliance Reviewer — Architecture	1
Purpose	1
Components	1
The engine (epic-compliance)	2
Deployment & distribution	3
End-to-end flow	3
Reading results back (epic-api → epic-web)	4
Design principles	4
Known limitations / future work	4

EPIC Compliance Reviewer — Architecture

Purpose

The Compliance Reviewer is an EPIC pipeline gate that verifies a checked-out application repository against PG&E's own AIDLC steering docs — the *T&S R&C Unified Controls Framework* (NIST 800-53). It runs as the **Review** stage, after download and before build, and blocks the pipeline when an app fails a gating control.

It is a **peer of Wiz and SonarQube**, not a replacement:

Tool	Scope
Wiz	Cloud security posture
SonarQube	Code quality
Compliance Reviewer	PG&E-specific policy-as-code from the AIDLC steering docs, keyed to NIST control IDs

Its unique value is the PG&E steering docs no other tool knows about.

Components

Component	Tech	Role
epic-compliance	Go CLI (static binary)	The engine. Scans a repo, emits findings, gates via exit code.
S3 artifact bucket	pge-epic-compliance-reviewer (acct 514712703977)	Hosts the version-pinned binary. KMS-encrypted, versioned, org-scoped read.

Component	Tech	Role
Review stage	ADO YAML (epic-pipeline/review/main.yml)	Pulls + runs the binary on the self-hosted agent; exit code gates the build.
epic-api	.NET	Carries the review toggle to ADO; reads the Review stage status + report back for display.
epic-web	Angular	"Review App" toggle in New Run; Review column + downloadable report in run history.
Steering docs	projects/pge-aidlc/	Source of the control catalog + requirements.

The engine (epic-compliance)

A single self-contained Go binary. No runtime to pre-install; drops into the CI workspace and runs.

Control catalog — all **76** app-applicable NIST controls from the framework are catalogued, each tagged by how it can be evaluated:

- **~18 code-checkable** — a rule produces a real verdict from the repo.
 - **6 hard** (concrete criterion, e.g. AC-06 lockout $\leq 10/60$ min, AU-02 six audit-record fields) → **gate** the build.
 - **~12 presence** (mechanism-present-only, no threshold in the doc) → informational.
- **~58 not repo-checkable** (personnel, process, pen tests, or vague/empty requirement text) → auto-emitted as **MANUAL** so every report accounts for the full framework rather than silently hiding them.

Hybrid evaluation — two engines behind one CLI:

1. **Deterministic** — regex/heuristic checks over the source tree (fast, free, reproducible).
2. **LLM** — interpretive controls escalate to **Claude Opus 4.8 via the PG&E Portkey gateway** (Bedrock). The LLM reads the actual evidence and confirms or overturns the heuristic verdict, which materially cuts false positives (e.g. distinguishing real auth enforcement from prompt/comment text). One call per interpretive finding; retry-with-backoff + request pacing on the gateway; **fails open** to the deterministic verdict if the gateway errors, never crashing the gate.

Verdict model: PASS / PARTIAL / FAIL / N/A / MANUAL.

Outputs (one run produces all three):

- **SARIF 2.1.0** — for ADO Advanced Security.
- **JSON** — native schema for the EPIC dashboard / audit record.
- **Markdown** — human-readable report (downloadable from epic-web).

Gate contract — the process **exit code** is the gate, not any parsed file:

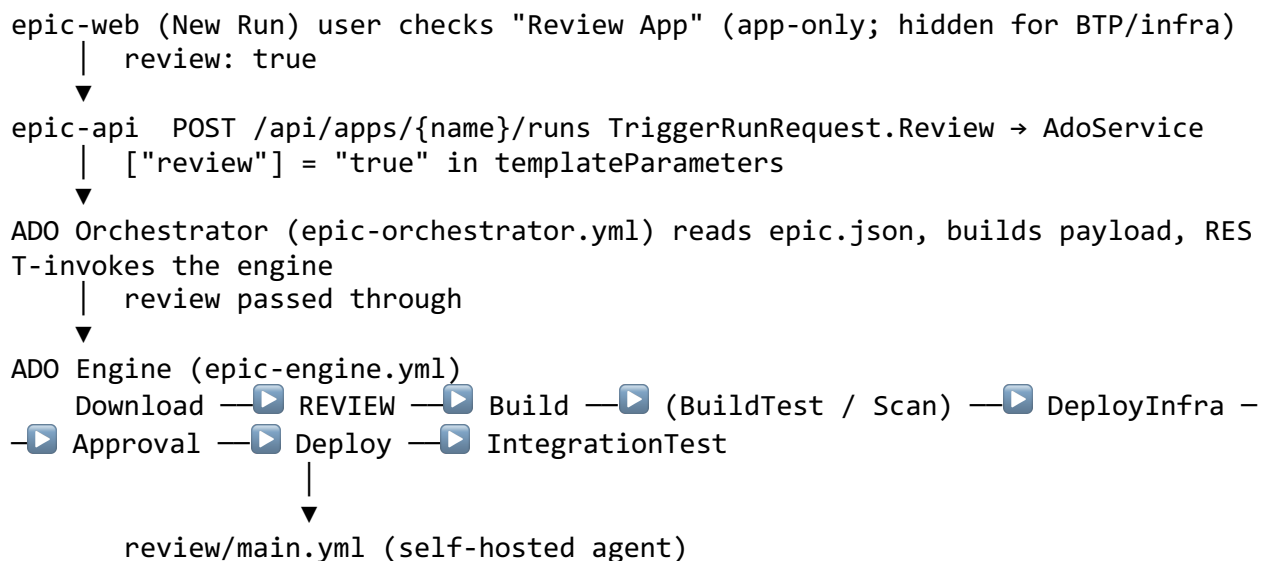
- 0 — pass (proceed)
- 1 — a gating (HARD) control failed → pipeline stops
- 2 — tool/runtime error

Policy is `--fail-on hard-fail` (default): only HARD-kind FAILs gate;
PARTIAL/MANUAL/N/A are informational.

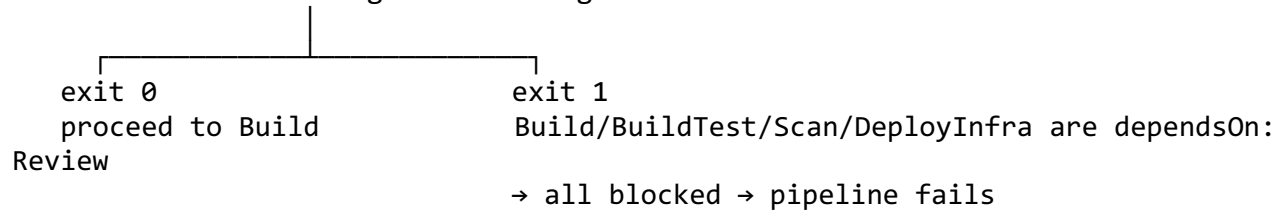
Deployment & distribution

- Built as a **version-pinned linux/amd64 binary** (`make release`) and published to S3: `s3://pge-epic-compliance-reviewer/compliance/epic-compliance-<version>-linux-amd64`.
- The **Review stage pulls it per-run into the cleaned workspace** (`aws s3 cp → chmod +x → run`). Because EPIC stages use `workspace: clean: all`, the binary is wiped every run — **fresh each time, no drift** on the shared agent.
- Runs on the **EPIC self-hosted agent** (the same box that runs SonarQube, Wiz, JFrog), which already has AWS creds + S3/KMS read on the bucket (granted by the EPIC AWS Resources/Compliance Reviewer/ Terraform stack).
- Version is controlled by the `COMPLIANCE_REVIEWER_VERSION` variable in the GV-account-access ADO Library group. Bump it to roll a release.
- Secrets (`PORTKEY_API_KEY`, `PORTKEY_BASE_URL`, `PORTKEY_MODEL`) come from the same variable group, injected into the run step's environment.

End-to-end flow



1. download epic-app source artifact
2. aws s3 cp the pinned epic-compliance binary
3. `./epic-compliance <src> --app-type X --llm --fail-on hard-fail --sarif ... --out ... --md ...`
4. publish reports as the "epic-compliance-review" build artifact
5. EXIT CODE gates the stage



Gate mechanics: Build, BuildTest, Scan, and DeployInfra each declare dependsOn: Review (when review=true). ADO's fail-fast on a failed dependency means one non-compliant Review halts the entire downstream pipeline.

Deploy/Approval/IntegrationTest require build=true, so they are transitively gated through Build.

Reading results back (epic-api → epic-web)

- **Stage status:** epic-api reads the ADO timeline, maps the Review/Review App stage to PipelineStages.Review, and persists a StageReview column on the run. epic-web renders a **Review** column (a status dot) in run history, positioned between Download and Build. Historical runs (pre-Review) show Skipped = "did not run".
- **Report download:** epic-web's Review stage detail has a "**Download Report**" button. It calls GET /api/apps/{name}/runs/{runId}/compliance-report, which has epic-api fetch the epic-compliance-review ADO build artifact (a zip), extract the .md, and return it; the browser downloads it client-side.

Design principles

1. **Focused, not a general reviewer** — one job: PG&E steering-doc compliance.
2. **Honest coverage** — the full 76-control framework is represented; the tool never fake-passes a control it can't verify (those are MANUAL).
3. **Deterministic where it matters** — a build gate needs reproducible verdicts; the LLM refines, it doesn't own the gate.
4. **No drift** — self-contained binary, version-pinned, workspace-scoped install wiped every run.
5. **Peer to existing tools** — same on-agent, exit-code-gated shape as Wiz/SonarQube; epic-api is a results sink, not the execution engine.

Known limitations / future work

- Detection heuristics are v1 (regex + LLM); deepening the deterministic rules is ongoing.

- The pge-aidlc SECURITY-XX numbering collision is unresolved, so findings key by NIST control ID only for now.
 - The -APP.md source is a degraded PDF extraction — a clean source is wanted before hard-coding more thresholds.
 - **Phase 2:** an IaC rule pack (Terraform against enterprise-standards/) plus the security-baseline.md app-code rules, dispatched by appType. v1 is app-focused.
-